5/6/2026

Applying GitHub Bug-Fix Classifiers to Evaluate Narrative–Technical Drift in Postmortems

Spasko Aleksov, Jimmy Jin

ECE 60872

Saurabh Bagchi

PURDUE UNIVERSITY

Overview

Introduction
Related Work
Our Design
Evaluation
Conclusion
Future Work

Introduction

Modern software systems fail in complex ways

Failures can be documented in many ways, we are specifically looking at:

- GitHub bug reports
- Pull requests
- Incident reports
- Public postmortems

These documents can contain useful engineering lessons

PURDUE UNIVERSITY

Introduction and Motivation

But failure narratives are not always purely technical

- Companies may want to reassure users
- Reports may protect reputation
- Public documents may avoid exposing full technical risk
- A bug may be described as minor even when the fix touches critical code
- Companies may want to appeal to shareholders

Introduction and Motivation [cont]

But failure narratives are not always purely technical

- Companies may want to reassure users
- Reports may protect reputation
- Public documents may avoid exposing full technical risk
- A bug may be described as minor even when the fix touches critical code
- Companies may want to appeal to shareholders

This creates the central problem:

Introduction and Motivation [cont]^2

But failure narratives are not always purely technical

- Companies may want to reassure users
- Reports may protect reputation
- Public documents may avoid exposing full technical risk
- A bug may be described as minor even when the fix touches critical code
- Companies may want to appeal to shareholders

This creates the central problem:

The story told about a failure may not match the technical evidence behind the fix

We call this mismatch narrative–technical drift

[Aside] Where we started:

Project #6: Comparing postmortems and practices

Every engineered system will encounter a failure -- even the fault-tolerant ones. Major tech companies put out postmortems and incident reports to describe what went wrong and how they plan to ameliorate it.

Goal:
You will develop a tool to automatically distinguish "engineering-oriented" from "politically-oriented" postmortem writeups across major technology companies, and describe the extent to which a public body of engineering-oriented postmortems truly exists.

Related Work

[1] Anandayuvaraj, Dharun, and James C. Davis. "Reflecting on Recurring Failures in IoT Development." 37th IEEE/ACM International Conference on Automated Software Engineering. 2022.

[2] Department of Homeland Security. “Cyber Safety Review Board Releases Unprecedented Report of its Review into Log4j Vulnerabilities and Response.” https://www.dhs.gov/news/2022/07/14/cyber-safety-review-board-releases-report-its-review-log4j-vulnerabilities-and

[3] Chen, Junjie, Xiaoting He, Qingwei Lin, Yong Xu, Hongyu Zhang, Dan Hao, Feng Gao, Zhangwei Xu, Yingnong Dang, and Dongmei Zhang. "An empirical investigation of incident triage for online service systems." In 2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), pp. 111-120. IEEE, 2019

PURDUE UNIVERSITY

Related Work: Learning from Software Failures and Postmortems

- Prior research studies recurring software failures, incident triage, and engineering lessons from postmortems
- IoT failure studies show that many failures repeat and can provide reusable engineering knowledge
- The DHS Log4j report demonstrates how government postmortems can document security impact, response failures, and ecosystem-level lessons
- Incident triage research studies how online service teams detect, prioritize, and respond to production failures
- These works motivate the need for automated methods that can compare failure descriptions against technical evidence at scale

Related Work: Gap

Prior work shows that software failures can repeat and provide reusable engineering lessons

Anandayuvaraj and Davis show that failure reports can reveal recurring patterns across systems

Existing work mainly studies:

- Types of failures
- Incident triage
- Lessons from postmortems

Related Work: Gap [cont]

Prior work shows that software failures can repeat and provide reusable engineering lessons

Anandayuvaraj and Davis show that failure reports can reveal recurring patterns across systems

Existing work mainly studies:

- Types of failures
- Incident triage
- Lessons from postmortems

Gap: Prior work does not directly measure whether the written narrative matches the technical evidence

Related Work: Contributions

Our contribution:

1. **Phase 1:**
   - Build classifiers from GitHub bug-fix reports
   - Use classifiers to determine drift
2. **Phase 2:**
   - Creation of Labeling rubric based on what is considered accountable framing
   - Manual Labelling and LLM Labeling Postmortems
3. **Phase 3:**
   - Compare the GitHub drift classifier signals from Phase 1 with the postmortem labels from Phase 2

Our Design: Three-Phase Pipeline

GitHub Bug-Fix Phase

- Collect bug reports and linked pull requests
- Extract narrative features from issue/PR text
- Extract technical features from code patches
- Compute narrative–technical drift scores

Postmortem Labeling Phase

- Create a rubric for accountable framing
- Manually label 15 postmortems
- Use LLMs to generate additional scoring signals

Transfer/Evaluation Phase

- Compare GitHub-derived classifier signals with postmortem labels
- Test whether bug-fix drift signals help evaluate postmortem quality
- Flag high-drift or low-accountability cases for manual inspection

GitHub Data Collection

Phase 1: GitHub Bug-Fix Classifiers

We collect recent closed GitHub issues and link them to merged pull requests.

Repositories used in the pipeline:

- python/cpython
- nodejs/node
- microsoft/vscode
- kubernetes/Kubernetes
- rust-lang/rust
- rails/rails
- golang/go
- Homebrew/homebrew-core

Collection method

Phase 1: GitHub Bug-Fix Classifiers

- Fetch recent closed issues
- Remove GitHub PRs from the issue list
- Use GitHub timeline cross-references to find linked PRs
- Fall back to GitHub search when timeline links are missing
- Keep only merged PRs
- Fetch files changed in the merged PR

Collected dataset:

- 752 recent closed issues checked
- 248 linked issue–PR rows collected

Linking Bug Reports to Fixes

Phase 1: GitHub Bug-Fix Classifiers

How We Link Narratives to Technical Fixes

- Each collected row connects a bug report narrative to the code patch that resolved it.

Narrative side:

- Issue title
- Issue body
- Issue labels
- PR title
- PR body

Technical side:

- Files changed
- Number of files changed
- Lines added
- Lines deleted
- Total lines changed
- Sensitive file indicators

Key Idea:

A linked issue–PR pair lets us compare what people said happened with what the patch changed

Feature Extraction

Phase 1: GitHub Bug-Fix Classifiers

Narrative features:

Count high-severity terms:

crash, security, vulnerability, exploit, data loss, outage, panic, corruption

Count low-severity terms:

typo, cleanup, minor, docs, refactor, formatting, style

Compute:

Narrative severity score =

high-severity hits − low-severity hits

Technical features:

Number of changed files

Total lines changed

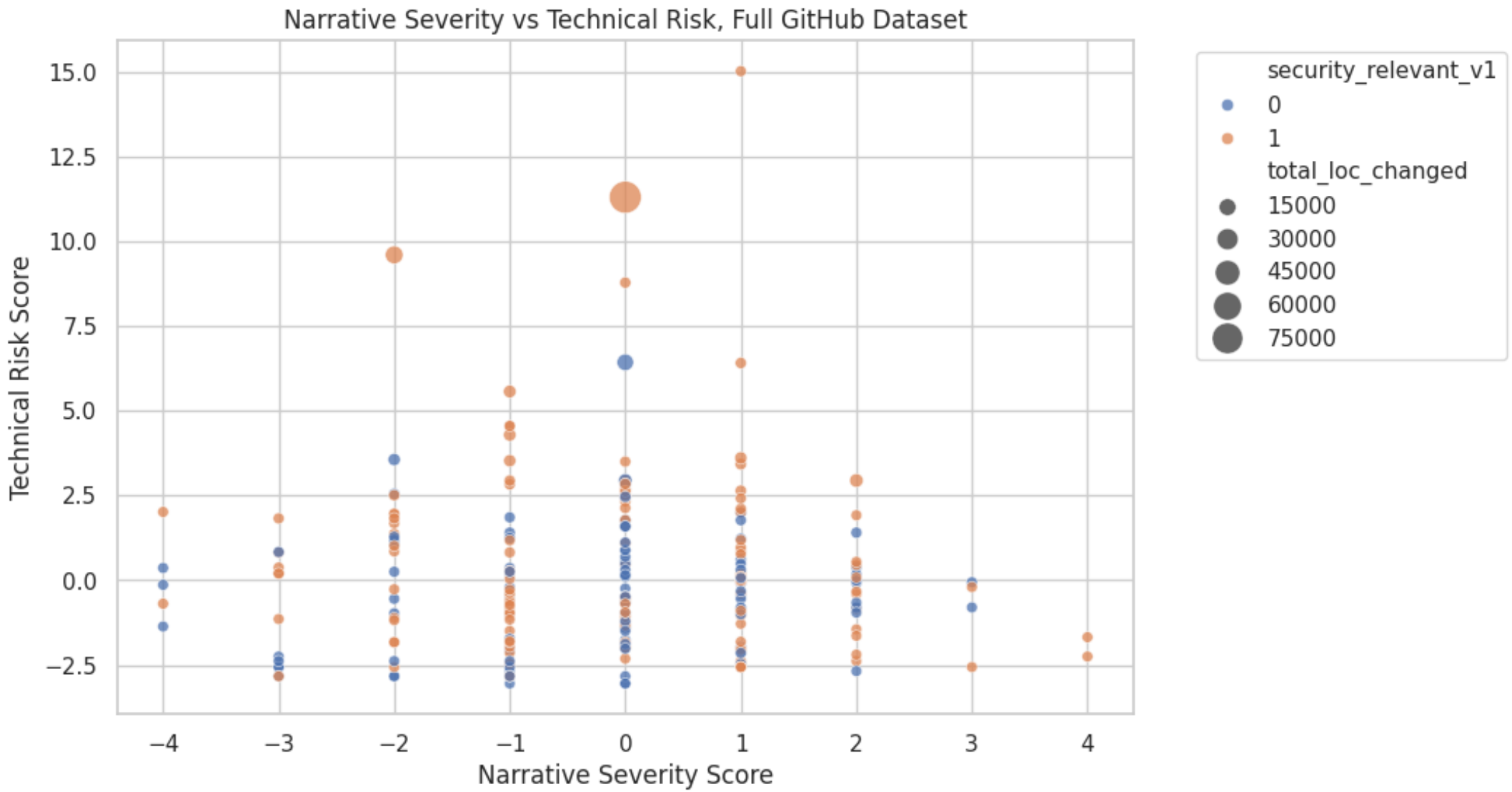
Sensitive file count

Sensitive file patterns:

auth, security, crypto, token, password, parser, network, memory, buffer, sandbox

Output scores:

- Narrative severity score
- Technical risk score
- Drift score
- Security-relevance label


Narrative Severity vs Technical Risk, Full GitHub Dataset
Technical Risk Score
15.0
12.5
10.0
7.5
5.0
2.5
0.0
−2.5
−4
−3
−2
−1
0
1
2
3
4
Narrative Severity Score
security_relevant_v1
0
1
total_loc_changed
15000
30000
45000
60000
75000

Narrative–Technical Drift Score

Phase 1: GitHub Bug-Fix Classifiers

We define drift as the mismatch between narrative severity and technical risk:

Drift score = z-score(technical risk score) − z-score(narrative severity score)

High drift means the issue–PR pair deserves manual inspection, not necessarily malicious.

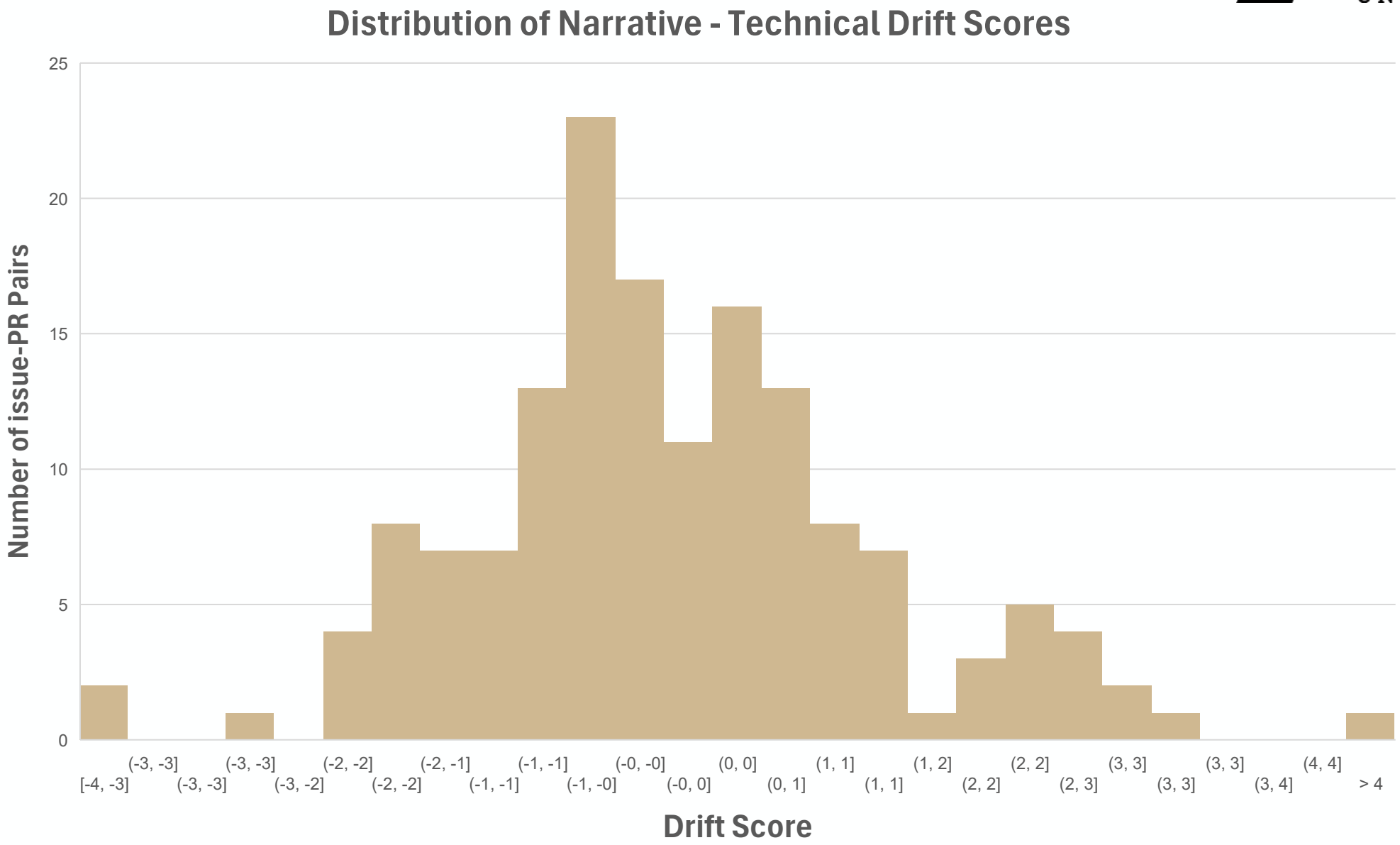
Observed in collected data:

248 issue–PR pairs collected

147 marked security-relevant by the first heuristic

Highest drift quartile had much larger patches on average:

- ~39.7 files changed
- ~2475 lines changed
- ~1.23 sensitive files per fix


Distribution of Narrative - Technical Drift Scores
Number of issue-PR Pairs
25
20
15
10
5
0
[-4, -3]
(-3, -3]
(-3, -3]
(-3, -3]
(-3, -2]
(-2, -2]
(-2, -2]
(-2, -1]
(-1, -1]
(-1, -1]
(-1, -0]
(-0, -0]
(-0, 0]
(0, 0]
(0, 1]
(1, 1]
(1, 1]
(1, 2]
(2, 2]
(2, 2]
(2, 3]
(3, 3]
(3, 3]
(3, 3]
(3, 4]
(4, 4]
> 4
Drift Score

Narrative–Technical Drift Score [cont]

Phase 1: GitHub Bug-Fix Classifiers

Example of High-Drift Issue–PR Pairs

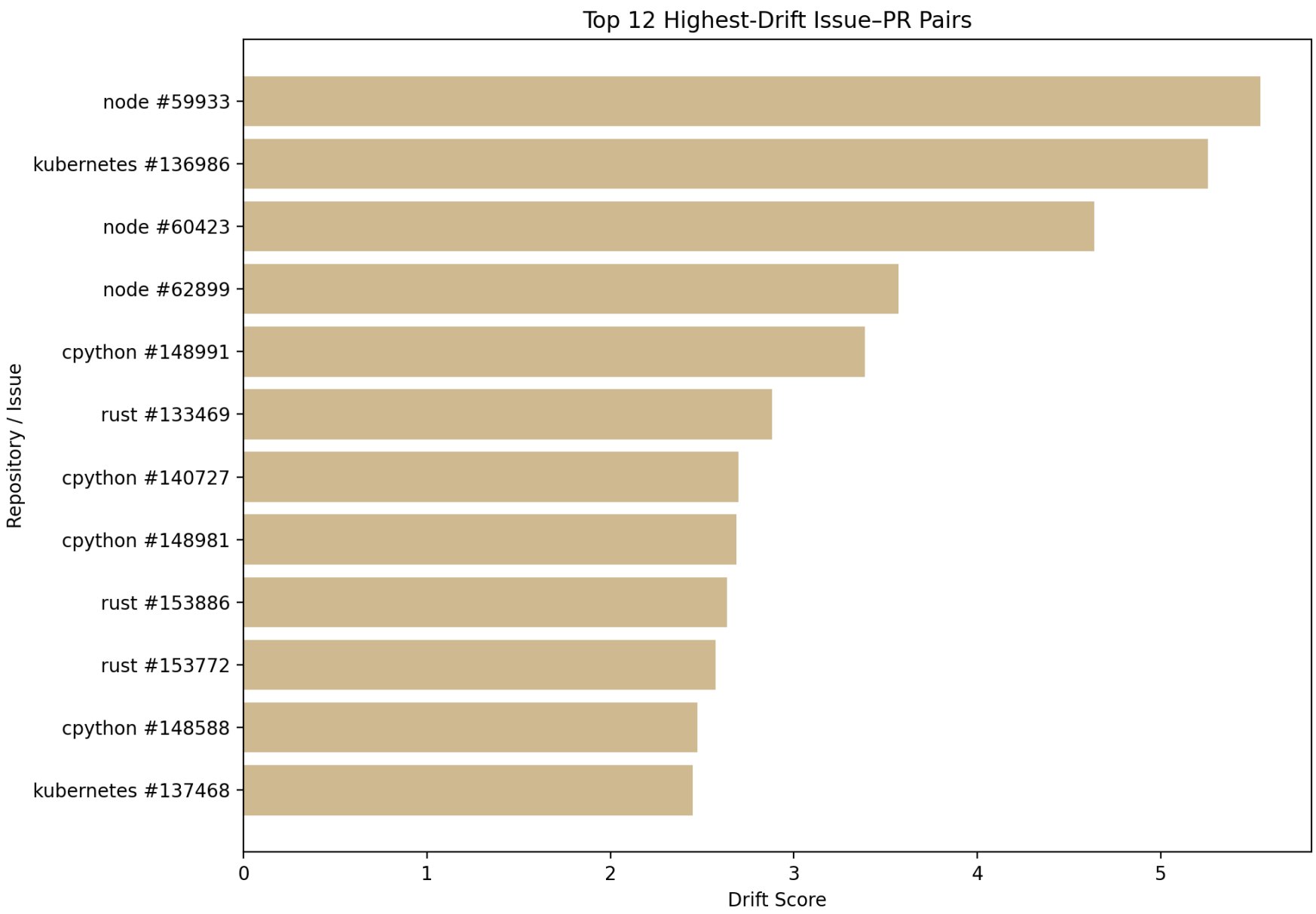
High drift often appears when the issue/PR language is not very severe, but the patch is technically large or sensitive

nodejs/node

- Issue: “Use C:/ path searching in windows over C:\”
- PR: V8 dependency cherry-pick
- 58 files changed
- 21 sensitive file matches

kubernetes/kubernetes

- Issue: Umbrella KEP API change
- PR: Create Workload API
- 248 files changed
- 18,004 lines changed


Top 12 Highest-Drift Issue–PR Pairs
node #59933
kubernetes #136986
node #60423
node #62899
cpython #148991
rust #133469
cpython #140727
cpython #148981
rust #153886
rust #153772
cpython #148588
kubernetes #137468
Repository / Issue
0
1
2
3
4
5
Drift Score

Overview: Postmortem Accountability Labeling

Phase 2: Postmortem Accountability Labeling

Goal of Phase 2

Evaluate whether public postmortems represent accountability in a clear and technical way

We score how strongly the postmortem text makes the failure:

- **Legible** — readers can understand what happened
- **Attributable** — readers can see what caused it
- **Repairable** — readers can see what will be fixed

Phase 2 creates the postmortem scores that are later compared against GitHub drift signals

Dataset

Phase 2: Postmortem Accountability Labeling

Postmortem Dataset

We selected **15 public postmortems** focused on broad technology incidents

Each postmortem is graded using the same accountability rubric

Each document receives:

- Manual score
- ChatGPT 5.5 score
- Claude Opus 4.7 score

The goal is to compare human judgment with LLM-assisted rubric scoring

At the end of Phase 2, we compute an **arithmetic mean score** across graders

Accountability Rubric

Phase 2: Postmortem Accountability Labeling

Metric	What it measures	Possible Score
1. Quantification of User Impact	Does the report quantify harm?	0 - 2
2. Internal-Awareness Timeline	Can readers reconstruct what was known and when?	0 - 2
3. Causal Specificity / Agency	Does the report identify concrete causes?	0 - 2
4. Failure Enumeration	Does it name the broader failure stack?	0 - 2
5. Action Item Concreteness	Are fixes specific and verifiable?	0 - 2
6. Stance Toward Harm Caused	Does it acknowledge harm or responsibility?	0 - 2
7. Framing Subject	Does the report center the failure itself?	0 - 1

Rubric Score Meaning

Phase 2: Postmortem Accountability Labeling

Each postmortem receives a total accountability-framing score out of **13**.

Score Range	Interpretation
9 - 13	High accountability framing
7 - 8	Mixed accountability framing
0 - 6	Low accountability framing

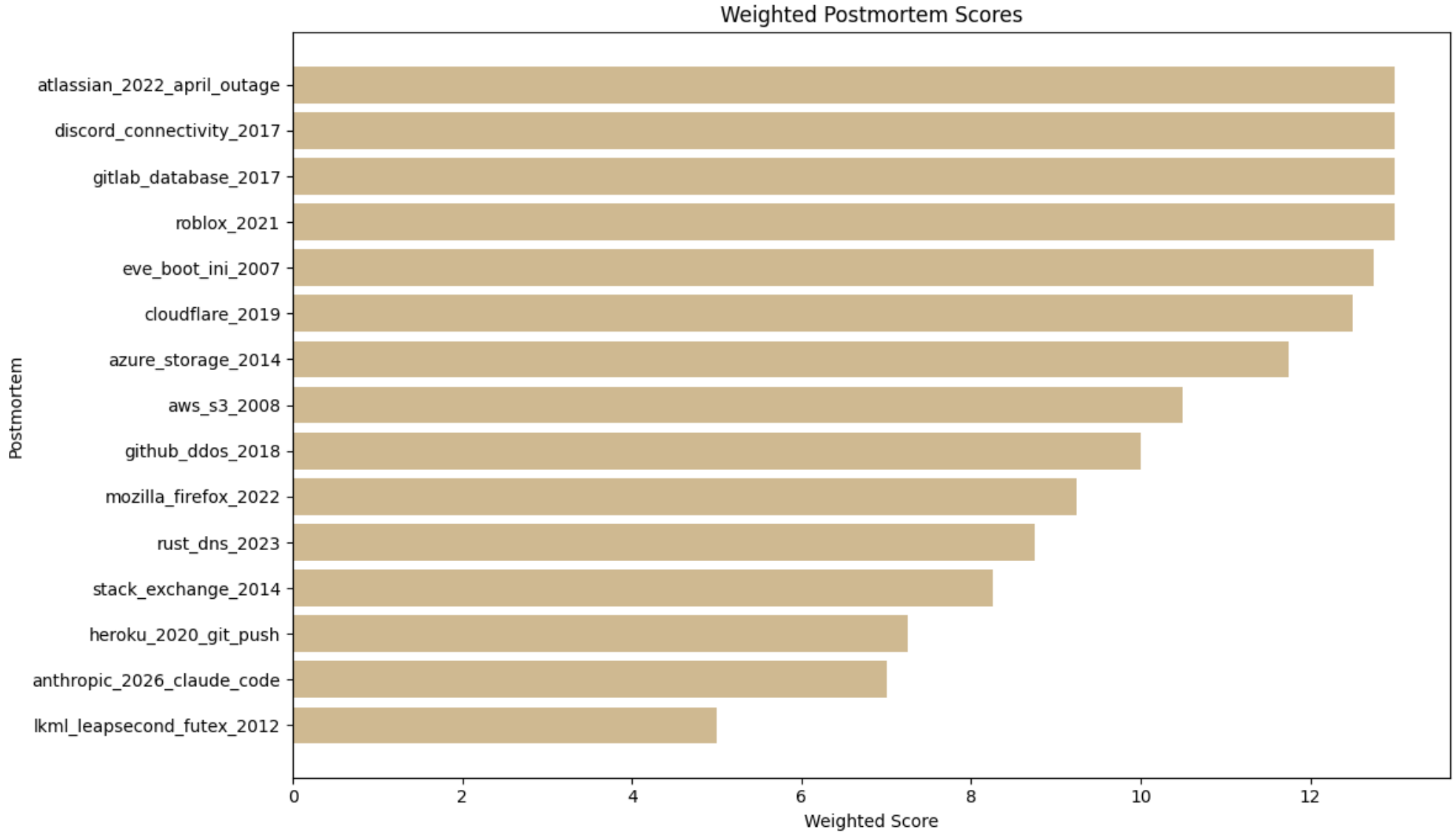
Manual and LLM Labeling Process

Phase 2: Postmortem Accountability Labeling

For each of the 15 postmortems:

- 2 Human Graders (Spasko and Jimmy) - > Consensus Results
- 2 LLMs (GPT 5.5 and Opus 4.7) -> 2

Finals score for a postmortem is weighted average of 3 scores


Weighted Postmortem Scores
atlassian_2022_april_outage
discord_connectivity_2017
gitlab_database_2017
roblox_2021
eve_boot_ini_2007
cloudflare_2019
azure_storage_2014
aws_s3_2008
github_ddos_2018
mozilla_firefox_2022
rust_dns_2023
stack_exchange_2014
heroku_2020_git_push
anthropic_2026_claude_code
lkml_leapsecond_futex_2012
Postmortem
0
2
4
6
8
10
12
Weighted Score

Connecting GitHub Drift to Postmortems

Phase 3: Transfer/Evaluation Phase

Goal:
Use GitHub drift patterns from Phase 1 to evaluate postmortem narratives from Phase 2.

Inputs from Phase 1:

- GitHub issue–PR scores
- Narrative wording patterns from each pair

Inputs from Phase 2:

- Manual postmortem scores
- LLM Scores
- Final averaged accountability score

Main question:
Can GitHub-derived drift signals help evaluate whether postmortems are engineering-oriented or narratively framed?

Phase 3 Method

Phase 3: Transfer/Evaluation Phase

Use Phase 1 drift scores to create weak labels

- Bottom 25% = low drift
- Top 25% = high drift
- Middle 50% excluded from classifier training

Train semantic drift classifier

- Embed GitHub issue–PR narratives using BAAI/bge-base-en-v1.5
- Train balanced logistic regression to classify high vs low drift

Apply classifier to postmortems

- Split postmortems into overlapping text chunks
- Predict high-drift probability for each chunk
- Aggregate chunk scores into a postmortem-level drift score

Compare to accountability scores

- Regress Phase 2 weighted accountability score against model-predicted high-drift probability

Output and Evaluation

Phase 3: Transfer/Evaluation Phase

Model output for each postmortem:

- Predicted drift category
- High-drift similarity score
- Low-drift similarity score
- Matched wording or sentence-level evidence

Phase 2 Accountability Score	Expected Phase 3 Drift Signal
High score	Low drift
Mixed score	Mixed drift
Low score	High drift

Final result:
We inspect whether GitHub-derived drift predictions agree with our manual and LLM postmortem scores

Evaluation

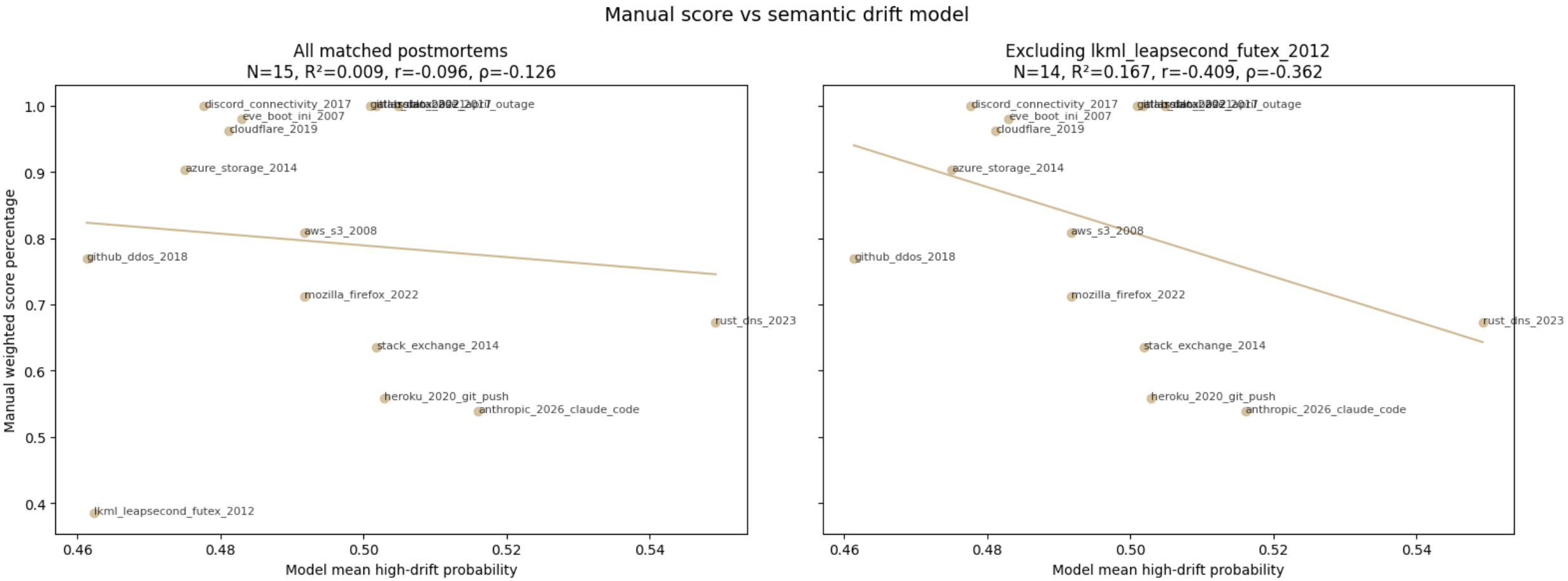
The early results are … to be expected
Current postmortem dataset is small:

- Only 15 postmortems
- Only 7 repositories
- 248 pull requests

A good next step is expanding the dataset:

More postmortems
More graders for phase 2
More (diverse) GitHub repositories
More Pull Requests

The current results are preliminary, but the pipeline provides a starting point for testing whether narrative–technical drift can generalize from GitHub bug fixes to public postmortems.


Manual score vs semantic drift model
All matched postmortems
N=15, R²=0.009, r=-0.096, ρ=-0.126
Excluding lkml_leapsecond_futex_2012
N=14, R²=0.167, r=-0.409, ρ=-0.362
Manual weighted score percentage
Model mean high-drift probability
discord_connectivity_2017
eve_boot_ini_2007
cloudflare_2019
azure_storage_2014
aws_s3_2008
github_ddos_2018
mozilla_firefox_2022
rust_dns_2023
stack_exchange_2014
heroku_2020_git_push
anthropic_2026_claude_code
lkml_leapsecond_futex_2012
1.0
0.9
0.8
0.7
0.6
0.5
0.4
0.46
0.48
0.50
0.52
0.54

Conclusion and Future Work

We built a three-phase exploratory pipeline for utilizing narrative-technical drift which contains:

- GitHub bug fix classifier
- Pseudo-manual postmortem accountability scoring
- Transfer comparison between GitHub drift and postmortem labels

To continue the project, we would recommend exploring:

- Increasing the stability and robustness of the classifier
- Getting more grounded/specialized graders for phase two or automate the process
- Evaluate different reliable rubrics and explore other potentially object extractable metrics from github
- Explore parallel studies in non-tech adjacent industry spaces

Discussion

Q & A

PURDUE UNIVERSITY

THANK YOU

PURDUE UNIVERSITY®